

A memory engine for AI agents.

MemoryAgent

Engram decides what an agent should **remember**, what to **recall under a hard token budget**, what to **forget on schedule**, and what to **stop believing** when new information contradicts old.

LIVE engram.orionbot.online · REPO github.com/OrionArchitekton/engram (MIT) · VIDEO [see Devpost submission](#)

01 The problem

Agents forget everything between sessions, or "solve" it by stuffing the whole transcript into context: cost grows without bound, stale facts linger forever, and contradictory facts coexist with equal confidence. The track brief names the three hard parts, and most memory demos ship only the first: efficient storage and retrieval, *timely forgetting*, and *recall within limited context windows*. Engram implements all three, and makes every decision observable.

02 One turn through the engine

```
Browser / MCP client  Next.js 15 + MCP stdio server (same engine, same store)
| POST /api/chat { sessionId, message } (rate-limited, key server-side)
▼
1. Decay sweep      retention = 0.5^(age / half-life(type) × importance) < 0.05 → decayed (never deleted)
2. Recall           score = 0.5 sim + 0.2 retention + 0.2 importance + 0.1 frequency
                    greedy-packed under a HARD 800-token budget (property-tested: never over)
3. Answer           qwen3.7-plus, temperature 0, recalled memories in system prompt
4. Write            LLM importance gate → nearest neighbors ≥ 0.45 sim → qwen3.7-plus
                    adjudicates conflict vs compatible → old fact SUPERSEDED with audit pointer
▼
SQLite  content · 256-dim text-embedding-v4 vector · type · importance · access · status
```

03 The differentiator: forgetting and truth maintenance

Selection is table stakes; the two behaviors most memory agents skip are the ones you can watch live on Engram's memory board:

DECAYED

An unused low-importance episode falls below the retention floor and silently leaves recall. Preserved for audit, never deleted.

SUPERSEDED

"I just moved to Denver" retires "I live in San Diego": LLM-adjudicated, pointer to the replacement, verified live.

NOT STORED

Questions and small talk score low importance and are deliberately not stored. The board shows the decision either way.

04 Engineering rigor

- **67 unit tests, zero network.** The engine core is pure TypeScript with injected clock, embedder, and adjudicator. The budget packer carries a property-style test over 50 randomized pools: the selected set never exceeds the budget.
- **Fail-safe LLM plumbing.** Every model output that feeds a decision passes a defensive parser; a parser failure can never supersede a memory (defaults: importance 0.5, no conflict, no drafts). Temperature 0 everywhere.
- **Adversarially reviewed.** An independent review pass refuted 75 discrete claims in these materials against the code, tests, git history, and live probes; every finding was fixed or reworded, including a live-verified retune of the conflict-similarity gate.
- **MCP round-trip proven over the real protocol** (docs/submission/mcp-roundtrip.txt): remember → recall returns the same record through the spawned stdio server.

typescript

next.js 15

qwen3.7-plus

text-embedding-v4

sqlite

mcp

docker

alibaba function compute

vitest

05 Honest demo, by construction

"Every reply, score, and event in the demo is a real output captured from live qwen3.7-plus + text-embedding-v4 calls through this app's own engine paths." — README, Demo mode. The decay scene's seed memory is backdated (disclosed) because real retention decay needs weeks; the mechanism that fires is the production code path.

06 Try it in 60 seconds

1. Open the live app	Tell it a preference: I prefer TypeScript with strict mode. Watch STORED land on the board.
2. Start a new session	Click + new , ask Which language should I use? The answer uses the memory; the board shows RECALLED with its score.
3. Contradict yourself	Say Actually, I now prefer Python. The old preference flips to SUPERSEDED with a pointer to its replacement.

Dan Mercede · dan@danmercede.com · Built with Claude Code as implementation copilot (Co-Authored-By trailers in git log) · MIT · Backend on Alibaba Cloud Function Compute (Singapore), models on Qwen Cloud (DashScope Intl).